

🌙 Moon Market explicado para humanos

Guía completa para entender tu primer proyecto web 🖥️✨

🌙 Introducción

Moon Market es una mini tienda online creada para practicar desarrollo web desde cero.

Aunque parezca una web simple, en realidad este proyecto incluye MUCHOS conceptos importantes de frontend moderno:

- estructura web
- diseño visual
- responsive móvil
- interactividad
- lógica de programación
- almacenamiento de datos
- Git y GitHub

El objetivo NO era hacer un Amazon real 😭

El objetivo era:

entender cómo piensa una página web moderna.

Y eso es exactamente lo que has empezado a aprender con este proyecto 🌙✨

🌙 ¿Cómo funciona realmente una web?

Antes de entender el código, hay que entender una cosa MUY importante 👁️

Una web normalmente tiene 3 partes:

Tecnología

HTML

CSS

JavaScript

Función

Estructura

Diseño

Interactividad

Piensa en ello así:

HTML 🦴

Es el esqueleto.

Crea:

- botones
- textos
- imágenes
- secciones
- menús

Sin HTML no existiría la web.

CSS 🎨

Es la apariencia visual.

Controla:

- colores
- tamaños
- posiciones
- animaciones
- responsive móvil

Sin CSS la web sería solo texto horrible JAJAJA 😭

JavaScript ⚡

Es el cerebro interactivo.

Hace posible:

- clicks
- carrito
- animaciones dinámicas
- guardar datos
- lógica

Sin JavaScript la web no reaccionaría.

Estructura del proyecto

```
moon-market/  
├── index.php  
├── README.md  
├── css/  
│   └── style.css  
├── js/  
│   └── app.js  
└── img/
```

¿Qué hace cada archivo?

index.php

Es el archivo principal.

Aquí viven:

- navbar
- productos
- carrito
- footer
- botones
- textos

Piensa:

“aquí construimos físicamente la web”.

style.css

Es el archivo visual.

Aquí controlamos:

- colores
- tamaños
- sombras
- efectos

- responsive
- animaciones

Piensa:

“aquí hacemos que la web se vea bonita”.



Es el archivo de lógica.

Aquí ocurre:

- abrir carrito
- cerrar carrito
- añadir productos
- eliminar productos
- guardar datos

Piensa:

“aquí ocurre la magia interactiva”.



HTML funciona usando etiquetas.

Las etiquetas son instrucciones.

Ejemplo:

`<h1>Hola</h1>`

Eso significa:

“crea un título grande que diga Hola”.



<nav>

<h1> Moon Market</h1>

Inicio

Productos

<li id="cart-button">

 Carrito (0)

</nav>



<nav>

Le dice al navegador:

“esto es una barra de navegación”.

<h1>

Crea el título principal.

Crea una lista.

Cada elemento individual de la lista.

id="cart-button"

Le pone un nombre único al carrito.

Eso sirve para que JavaScript pueda encontrarlo después.

`<span>`

Es una mini zona de texto.

Aquí la usamos para mostrar:

0

Y luego JavaScript cambia ese número automáticamente.

Hero Section

`<section class="hero">`

`<h2>Descubre productos mágicos ✨</h2>`

`<p>Tu tienda aesthetic favorita.</p>`

`<button>Comprar ahora</button>`

`</section>`

Traducción humana

`<section>`

Divide la web en bloques organizados.

`class="hero"`

Le pone una etiqueta al bloque.

Las clases sirven para:

- encontrar elementos con CSS
 - encontrar elementos con JS
 - reutilizar estilos
-

`<button>`

Crea un botón.

Productos

```
<div class="card">
```

```
  
```

```
  <h3>Velas Lunares</h3>
```

```
  <p>19,99€</p>
```

```
  <button class="add-cart">
```

```
    Añadir al carrito
```

```
  </button>
```

```
</div>
```

Traducción humana

```
<div>
```

Es una caja contenedora.

Piensa:

“un bloque donde guardamos cosas”.

```
<img>
```

Muestra una imagen.

```
src="..."
```

La dirección de la imagen.

```
alt="Velas"
```

Texto alternativo por accesibilidad.

<h3>

Nombre del producto.

<p>

Texto normal.

Aquí lo usamos para el precio.

class="add-cart"

Le pone una etiqueta al botón.

Luego JavaScript usa esa clase para detectar TODOS los botones del carrito.

CSS explicado fácil

¿Qué es CSS?

CSS sirve para dar estilos.

Con CSS podemos:

- mover cosas
 - cambiar colores
 - crear animaciones
 - hacer responsive
 - añadir sombras
-

Fondo oscuro

```
body {  
  background: #0a0a18;  
  color: white;  
}
```

Traducción humana

background

Color del fondo.

color

Color del texto.

Flexbox

```
.products {  
  display: flex;  
  justify-content: center;  
  gap: 30px;  
}
```

Traducción humana

display: flex

Activa Flexbox.

Flexbox sirve para:

- alinear elementos
- organizarlos
- distribuir espacios

Es MUY importante en frontend moderno.

justify-content: center

Centra los productos.

gap

Espacio entre elementos.

Hover

```
.card: hover {  
  transform: translateY(-10px);  
}
```

Traducción humana

Cuando pasas el ratón: ✨ la tarjeta flota hacia arriba.

Animaciones

```
.card {  
  animation: fadeIn 1s ease;  
}
```

Traducción humana

Le dice:

“aparece suavemente”.

Responsive Design

```
@media (max-width: 768px)
```

Traducción humana

Significa:

“si la pantalla es pequeña, cambia el diseño”.

Gracias a eso la web funciona bien en móvil 📱

JavaScript explicado MUY fácil

¿Qué es JavaScript?

JavaScript es el lenguaje que hace que la web reaccione.

Sin JavaScript:

- el carrito no funcionaría
 - los botones no harían nada
 - no habría interactividad
-

Buscar botones

```
const buttons = document.querySelectorAll(".add-cart");
```

Traducción humana

Busca TODOS los elementos que tengan:

`.add-cart`

o sea:

todos los botones de añadir al carrito.

Variables

```
let count = 0;
```

Traducción humana

Creamos una variable llamada:

`count`

para guardar el número de productos.

Eventos

```
button.addEventListener("click", () => {  
  
});
```

Traducción humana

Significa:

“cuando alguien haga click...”

entonces ejecuta código.

Sumar productos

```
count++;
```

Traducción humana

Suma:

+1

al carrito.

Actualizar contador

```
cartCount.textContent = count;
```

Traducción humana

Cambia el número visible del carrito.

Añadir HTML dinámicamente

```
cartItems.innerHTML += `  
  
<div class="cart-item"></div>  
  
`;  
`;
```

Traducción humana

JavaScript crea HTML nuevo automáticamente.

Esto es MUY importante.

Se llama:

✨ manipulación del DOM.

LocalStorage

```
localStorage.setItem("moonCart", JSON.stringify(cart));
```

Traducción humana

Guarda el carrito en la memoria del navegador.

Así:

- recargas
 - y el carrito sigue ahí.
-

JSON.stringify

```
JSON.stringify(cart)
```

Traducción humana

Convierte arrays y objetos en texto.

Eso es necesario porque localStorage solo puede guardar texto.

¿Qué es el DOM?

El DOM es:

la representación de la web que JavaScript puede modificar.

Por eso podemos:

- cambiar textos
 - añadir productos
 - borrar elementos
 - actualizar números
-

¿Qué es Git?

Git sirve para controlar versiones.

Piensa:

“guardar puntos de restauración del proyecto”.

GitHub

GitHub sirve para:

- subir proyectos
 - compartir código
 - crear portfolio
 - colaborar
-

Comandos Git usados

Inicializar Git

```
git init
```

Crea un repositorio Git local.

Añadir archivos

```
git add .
```

Añade TODOS los archivos al seguimiento.

Crear commit

```
git commit -m "Primer commit"
```

Guarda una versión del proyecto.

Subir a GitHub

```
git push -u origin main
```

Sube el proyecto online.

Problemas reales que aparecieron

Y esto es IMPORTANTÍSIMO 👁

Porque programar NO es:

hacer código perfecto

Programar es:

resolver problemas constantemente

Durante el proyecto aparecieron:

- rutas mal escritas
- caché navegador
- IDs faltando
- CSS que no cargaba

- Git mal conectado
- elementos duplicados

Y eso es NORMAL 😭✨

Literalmente es desarrollo web real.

🌙 ¿Qué aprendiste realmente?

Con este proyecto aprendiste:

✅ HTML ✅ CSS ✅ Flexbox ✅ Responsive Design ✅ JavaScript ✅ Variables ✅
Eventos ✅ Arrays ✅ Objetos ✅ localStorage ✅ Manipulación DOM ✅ Git ✅
GitHub ✅ Debugging ✅ lógica frontend
